SOLID Model Binding Implementation

Overview

This document describes the SOLID-compliant model binding system implemented for the Chicken Processing Backend. The system separates concerns between validation, transformation, mapping, and binding while adhering to all five SOLID principles.

Architecture

1. DTOs (Data Transfer Objects)

Location: `src/dto/UserDTO.ts`

- **Purpose:** Define request/response data structures
- **SOLID Compliance:** SRP (each DTO has single responsibility)
- **Key Interfaces:**
  - `RegisterUserRequestDTO`, `LoginUserRequestDTO`, `UpdateUserRequestDTO`
  - `UserResponseDTO`, `AuthResponseDTO`
  - `IUserMapper`, `IUserValidator` (ISP)

2. Mappers

Location: `src/dto/mappers/UserMapper.ts`

- **Purpose:** Transform between DTOs and domain models
- **SOLID Compliance:** SRP (only handles mapping)
- **Key Class:** `UserMapper` implements `IUserMapper`

3. Validators

Location: `src/dto/validators/UserValidator.ts`

- **Purpose:** Validate request data using Joi schemas
- **SOLID Compliance:** SRP + ISP (specific validation interfaces)
- **Key Classes:**
  - `UserValidator` implements multiple `IValidator` interfaces
  - `ValidationError` for structured error handling
  - `ValidatorFactory` (Factory Pattern)

4. Transformers

Location: `src/dto/transformers/UserTransformer.ts`

- **Purpose:** Clean and format request/response data
- **SOLID Compliance:** SRP + ISP (specific transformation interfaces)
- **Key Classes:**
  - `UserRequestTransformer` for request data

- `UserResponseTransformer` for response data
- `TransformerFactory` (Factory Pattern)

5. Model Binders

Location: `src/dto/ModelBinder.ts`

- **Purpose:** Handle complete model binding flow
- **SOLID Compliance:** All five principles
- **Key Components:**
  - **Interfaces:** `IModelBinder<T>`, `IValidator<T>`, `ITransformer<From, To>` (ISP)
  - **Base Class:** `BaseModelBinder<T>` (OCP + SRP)
  - **Concrete Implementations:** `JsonModelBinder`, `QueryModelBinder`, `ParamsModelBinder` (LSP)
  - **Factory:** `ModelBinderFactory` (Factory Pattern + DIP)
  - **Utility:** `bindModel()` function for easy usage

6. Updated Controller

Location: `src/controllers/authControllerV2.ts`

- **Purpose:** Demonstrate SOLID model binding in practice
- **Key Features:**
  - Uses `ModelBinderFactory` to create binders
  - Uses `bindModel()` for request binding
  - Handles `ValidationError` gracefully
  - Separates concerns between binding and business logic

SOLID Principles Implementation

1. Single Responsibility Principle (SRP)

- **Validators** only validate
- **Transformers** only transform
- **Mappers** only map
- **Binders** only bind
- **Controllers** handle HTTP requests

2. Open/Closed Principle (OCP)

- `BaseModelBinder` can be extended for new binding types
- `UserValidator` can be extended with new validation rules
- `UserRequestTransformer` can be extended with new transformation logic
- New DTOs can be added without modifying existing code

3. Liskov Substitution Principle (LSP)

- `JsonModelBinder` can be used wherever `IModelBinder` is expected
- `UserValidator` can be used wherever `IValidator` is expected

- `UserRequestTransformer` can be used wherever `ITransformer` is expected

4. Interface Segregation Principle (ISP)

- `IValidator` has specific validation methods
- `ITransformer` has specific transformation methods
- `IModelBinder` has specific binding methods
- `IUserMapper` has specific mapping methods

5. Dependency Inversion Principle (DIP)

- High-level modules depend on abstractions (interfaces)
- `ModelBinder` depends on `IValidator` and `ITransformer`
- `Controller` depends on `IModelBinder`
- Factories create concrete implementations

Usage Example

In Controllers:

```
// Create binder
const registerBinder =
ModelBinderFactory.createJsonBinder<RegisterUserRequestDTO>(
  userValidator,
  userRequestTransformer
);

// Bind request
const registerData = await bindModel<RegisterUserRequestDTO>(req,
registerBinder);

// Use data
const result = await registerUser(registerData);
```

Creating Custom Binders:

```
// Custom validator
class CustomValidator implements IValidator<CustomDTO> {
  async validate(data: CustomDTO): Promise<void> {
    // Custom validation logic
  }
}

// Custom transformer
class CustomTransformer implements ITransformer<any, CustomDTO> {
  async transform(from: any): Promise<CustomDTO> {
    // Custom transformation logic
  }
```

```
}

// Create custom binder
const customBinder = ModelBinderFactory.createJsonBinder(
  new CustomValidator(),
  new CustomTransformer()
);
```

Benefits

1. **Separation of Concerns:** Clear boundaries between validation, transformation, mapping, and binding
2. **Testability:** Each component can be tested in isolation
3. **Maintainability:** Changes to one component don't affect others
4. **Extensibility:** New binding types can be added easily
5. **Reusability:** Components can be reused across different endpoints
6. **Type Safety:** Full TypeScript support with proper typing

Testing

Test File: `src/test-model-binding.ts`

- Tests all SOLID principles compliance
- Tests complete binding flow
- Tests error handling
- Tests individual components

Run tests with:

```
cd chicken-processing-backend
npx ts-node src/test-model-binding.ts
```

Migration Guide

From Legacy Controllers:

1. Create DTOs for your endpoint
2. Create validator, transformer, and mapper if needed
3. Create model binder using factory
4. Update controller to use `bindModel()`
5. Handle `ValidationError` appropriately

Adding New Endpoints:

1. Define DTOs in appropriate DTO file
2. Extend existing validator or create new one

3. Extend existing transformer or create new one
4. Create mapper if needed
5. Create binder using factory
6. Update controller

Conclusion

The SOLID model binding implementation provides a robust, maintainable, and extensible system for handling request/response data in the Chicken Processing Backend. By adhering to SOLID principles, the system ensures clean separation of concerns, easy testing, and long-term maintainability.